TP1 : Découverte comparative de React Native et Flutter

Composants • État • Interaction • Saisie • Géolocalisation

1. Objectifs et organisation du TP

1.1 Compétences visées

- Installer et valider un environnement de développement mobile pour React Native et Flutter.
- Créer une première application React Native en JSX et comprendre View, Text, Button, TextInput et StyleSheet.
- Manipuler les notions de props et de state dans React Native.
- Créer une application Flutter et distinguer StatelessWidget et StatefulWidget.
- Mettre à jour une interface Flutter avec setState(), puis découvrir la séparation de la logique avec Cubit/BLoC.
- Accéder à un capteur de localisation avec une bibliothèque moderne dans les deux environnements.
- Comparer concrètement les deux approches de développement hybrides-natives.

1.2 Livrables attendus

- Projet React Native nommé TP2ReactNative.
- Projet Flutter nommé tp2_flutter.
- Captures d'écran des applications exécutées sur un émulateur ou un téléphone.
- Un court tableau comparatif React Native / Flutter complété par l'étudiant.
- Les réponses aux questions de synthèse en fin de TP.

2. Rappels du cours avant de commencer

Notion	React Native	Flutter
Langage principal	JavaScript / JSX	Dart
Unité de construction UI	Component	Widget
État local	state + setState()	State + setState()
Style / mise en page	StyleSheet + Flexbox	Widgets de layout + Theme
Architecture d'état avancée	Flux / Redux (cours)	BLoC / Cubit
Capteurs	Modules / packages natifs	pub packages / Platform Channels

3. Installation — environnement commun

3.1 Logiciels à installer

Logiciel	Utilité	Vérification
Visual Studio Code	Éditeur pour JavaScript/JSX et Dart	Ouvrir VS Code
Git	Gestion de versions et certains outils de dépendances	git --version
Android Studio	Android SDK, Platform Tools et émulateur	SDK Manager / Device Manager
Un navigateur moderne	Tests Expo Web et documentation	Chrome / Edge / Firefox

3.2 Préparer Android Studio

1. Installer Android Studio.
2. Ouvrir SDK Manager et installer une version récente du Android SDK ainsi que Android SDK Platform-Tools.
3. Ouvrir Device Manager et créer un téléphone virtuel (par exemple un Pixel).
4. Démarrer l'émulateur une première fois avant la séance.
5. Optionnel : connecter un téléphone Android en activant les Options pour les développeurs et le Débogage USB.

Commande de contrôle si ADB est disponible :

```
adb devices
```

Conseil — Un seul émulateur suffit

Le même Android Studio / Android Emulator peut servir pour React Native et Flutter. Expo permet aussi de tester sur un téléphone réel avec Expo Go, ce qui simplifie fortement la première partie.

PARTIE A — React Native avec Expo

4. Installation de React Native / Expo

4.1 Installer Node.js LTS

- Installer une version LTS de Node.js depuis le site officiel.
- Fermer puis rouvrir PowerShell après l'installation.
- Vérifier Node.js et npm avec les commandes suivantes.

```
node -v
npm -v
```

4.2 Créer le projet

```
npx create-expo-app@latest TP2ReactNative --template blank
cd TP2ReactNative
npx expo start
```

- Sur téléphone : installer Expo Go, puis scanner le QR code affiché dans le terminal.
- Sur émulateur Android : démarrer l'émulateur puis utiliser la commande/option Android proposée par Expo.
- Sur le Web : utiliser l'option web pour un test rapide des exercices sans capteur natif.

5. Exercice RN1 — Hello World et StyleSheet

Objectif

Créer une interface minimale et identifier les rôles de View, Text et StyleSheet.

Fichier App.js

```
import React, { Component } from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default class App extends Component {
  render() {
    return (
      <View style={styles.container}>
        <Text style={styles.title}>Hello, world!</Text>
        <Text>Mon premier écran React Native</Text>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  title: {
    fontSize: 26,
    fontWeight: 'bold',
    marginBottom: 12,
  },
});
```

Explication

Élément	Rôle
Component	Classe React qui possède une méthode render().
View	Conteneur d'interface, comparable à une zone de mise en page.
Text	Affiche du texte.
JSX	Syntaxe qui mélange JavaScript et balises de composants.
StyleSheet	Regroupe les propriétés de style proches du CSS/Flexbox.
flex: 1	Permet au conteneur d'occuper l'espace disponible.

Travail étudiant

1. Changer le texte principal par votre nom et groupe.
2. Ajouter un second Text affichant la date du TP.
3. Modifier l'alignement pour constater l'effet de justifyContent et alignItems.

6. Exercice RN2 — Bouton, événement et Alert

App.js — version actualisée de l'exemple du cours

```
import React, { Component } from 'react';
import { Alert, Button, StyleSheet, View } from 'react-native';

export default class App extends Component {
  testFunction = () => {
    Alert.alert('Interaction', 'Button Pressed !!!');
  };

  render() {
    return (
      <View style={styles.container}>
        <Button
          onPress={this.testFunction}
          title="Test Button"
          color="#009688"
        />
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    margin: 16,
  },
});
```

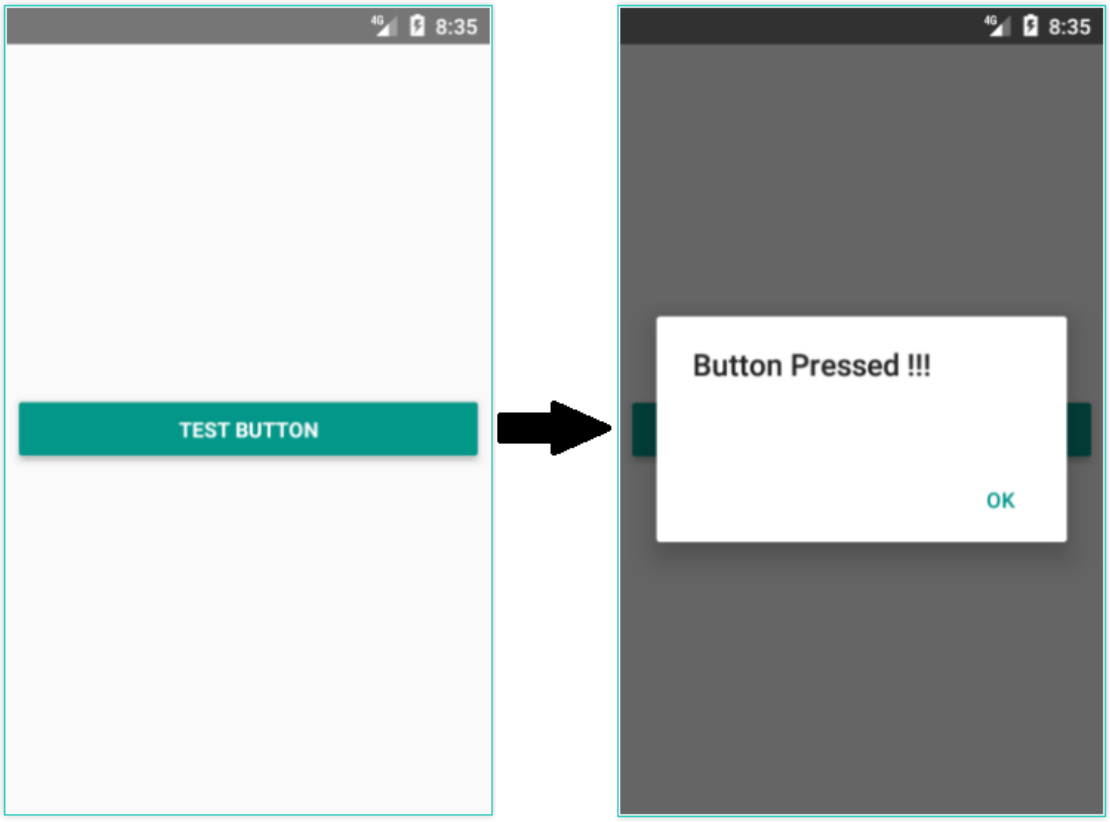


Figure — Sortie de l'exemple Button fournie avec le code de cours.

À comprendre

- **onPress :** propriété événementielle appelée lorsque l'utilisateur appuie sur le bouton.
- **Alert.alert() :** affiche une boîte de dialogue native simple.
- **Fonction fléchée :** elle évite ici d'avoir à utiliser bind(this) pour préserver le contexte de la classe.

Mini-défi

Ajouter un deuxième bouton « Annuler » qui affiche un autre message.

7. Exercice RN3 — État (state) et compteur

App.js

```
import React, { Component } from 'react';
import { Button, StyleSheet, Text, View } from 'react-native';

export default class App extends Component {
  constructor(props) {
    super(props);
    this.state = { counter: 0 };
  }

  incrementCount = () => {
    this.setState((previous) => ({
      counter: previous.counter + 1,
    }));
  };

  decrementCount = () => {
    this.setState((previous) => ({
      counter: previous.counter - 1,
```

```
    }));
  };

  render() {
    return (
      <View style={styles.container}>
        <Text style={styles.counter}>Count: {this.state.counter}</Text>
        <Button onPress={this.decrementCount} title="-" />
        <View style={{ height: 8 }} />
        <Button onPress={this.incrementCount} title="+" />
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', margin: 20 },
  counter: { fontSize: 24, textAlign: 'center', marginBottom: 20 },
});
```

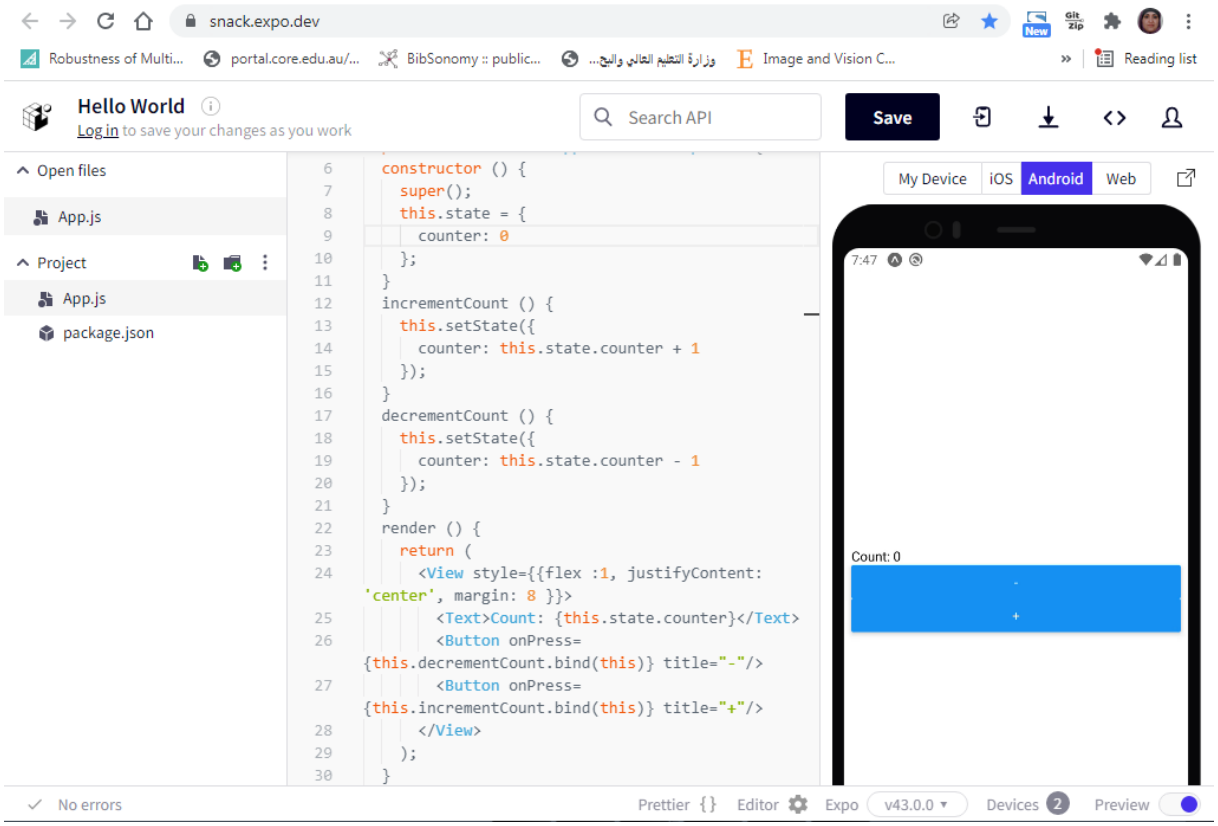


Figure — Exemple CounterState du support de cours exécuté dans Expo/Snack.

À retenir — Principe de réactivité

Ne modifiez jamais directement this.state.counter. Utilisez setState() : React recalcule ensuite l'interface concernée.

Travail étudiant

1. Ajouter un bouton Remettre à zéro.
2. Empêcher le compteur de devenir négatif.
3. Afficher un message « Valeur élevée » lorsque le compteur atteint 10.

8. Exercice RN4 — TextInput + state + bouton

App.js

```
import React, { Component } from 'react';
import { Alert, Button, StyleSheet, Text, TextInput, View } from 'react-native';

export default class App extends Component {
  state = { input: '' };

  handleChangeInput = (text) => {
    this.setState({ input: text });
  };

  showInput = () => {
    Alert.alert('Texte saisi', this.state.input || 'Aucune valeur');
  };

  render() {
    return (
      <View style={styles.container}>
        <Text>Saisir un texte :</Text>
        <TextInput
          style={styles.input}
          onChangeText={this.handleChangeInput}
          value={this.state.input}
          placeholder="Votre texte"
        />
        <Button onPress={this.showInput} title="OK" />
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 20 },
  input: {
    height: 44,
    borderColor: '#1F4E78',
    borderWidth: 1,
    borderRadius: 6,
    paddingHorizontal: 10,
    marginVertical: 12,
  },
});
```

Questions

- À quel moment handleChangeInput() est-elle appelée ?
- Pourquoi la propriété value dépend-elle de this.state.input ?
- Que se passe-t-il si on enlève onChangeText ?

9. Exercice RN5 — Props : communication parent → enfant

App.js

```
import React, { Component } from 'react';
import { StyleSheet, Text, View } from 'react-native';

class Heading extends Component {
  render() {
    return <Text style={styles.heading}>{this.props.message}</Text>;
  }
}
```

```
export default class App extends Component {
  render() {
    return (
      <View style={styles.container}>
        <Heading message="Titre reçu du composant parent" />
        <Heading message="Deuxième utilisation du même composant" />
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 20 },
  heading: { fontSize: 20, fontWeight: 'bold', marginVertical: 8 },
});
```

Concept	Interprétation
props	Données transmises au composant par son parent.
state	Données internes qui peuvent évoluer pendant l'exécution.
Réutilisation	Le même composant Heading est utilisé plusieurs fois avec des valeurs différentes.

10. Exercice RN6 — Accéder à la position GPS (adaptation moderne)

10.1 Installer le module

```
npx expo install expo-location
```

10.2 Code

App.js

```
import React, { Component } from 'react';
import { Button, StyleSheet, Text, View } from 'react-native';
import * as Location from 'expo-location';

export default class App extends Component {
  state = {
    latitude: null,
    longitude: null,
    message: 'Position non demandée',
  };

  getPosition = async () => {
    const permission = await Location.requestForegroundPermissionsAsync();

    if (permission.status !== 'granted') {
      this.setState({ message: 'Permission de localisation refusée' });
      return;
    }

    const position = await Location.getCurrentPositionAsync({});

    this.setState({
      latitude: position.coords.latitude,
      longitude: position.coords.longitude,
      message: 'Position obtenue',
    });
```

```
  };

  render() {
    return (
      <View style={styles.container}>
        <Button title="Obtenir ma position" onPress={this.getPosition} />
        <Text style={styles.info}>{this.state.message}</Text>
        <Text>Latitude : {this.state.latitude ?? '--'}</Text>
        <Text>Longitude : {this.state.longitude ?? '--'}</Text>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 24 },
  info: { marginTop: 18, marginBottom: 8, fontWeight: 'bold' },
});
```

Attention — Permission avant capteur

La demande d'autorisation doit précéder la lecture de la position. Sur un émulateur, configurez une position simulée ; sur un téléphone, activez le service de localisation.

Extension facultative

Utiliser Location.watchPositionAsync() pour observer des changements de position et comparer avec le tracking réalisé auparavant sous Ionic/Capacitor.

PARTIE B — Flutter et Dart

12. Installation de Flutter

12.1 Installer le SDK Flutter

1. Télécharger le SDK Flutter stable pour votre système.
2. Décompresser Flutter dans un dossier simple et accessible, par exemple C:\src\flutter.
3. Ajouter C:\src\flutter\bin à la variable PATH de Windows.
4. Fermer puis rouvrir PowerShell.
5. Installer les extensions Flutter et Dart dans Visual Studio Code.

Vérification :

```
flutter --version
dart --version
flutter doctor -v
```

12.2 Valider Android

```
flutter doctor --android-licenses
flutter devices
```

Contrôle — Objectif de flutter doctor

Avant de coder, corriger les erreurs signalées pour Android Toolchain et Android Studio. Les avertissements concernant des plateformes non utilisées peuvent être traités plus tard.

13. Créer le projet Flutter

```
flutter create tp2_flutter
cd tp2_flutter
code .
flutter run
```

Élément	Rôle
lib/main.dart	Point d'entrée principal de l'application.
pubspec.yaml	Dépendances, assets, métadonnées du projet.
android/	Projet natif Android généré.
test/	Tests Flutter.

14. Exercice FL1 — Hello World avec StatelessWidget

lib/main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'TP2 Flutter',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ),
        body: const Center(
          child: Text(
            'Hello World',
            style: TextStyle(fontSize: 28),
          ),
        ),
      ),
    );
  }
}
```

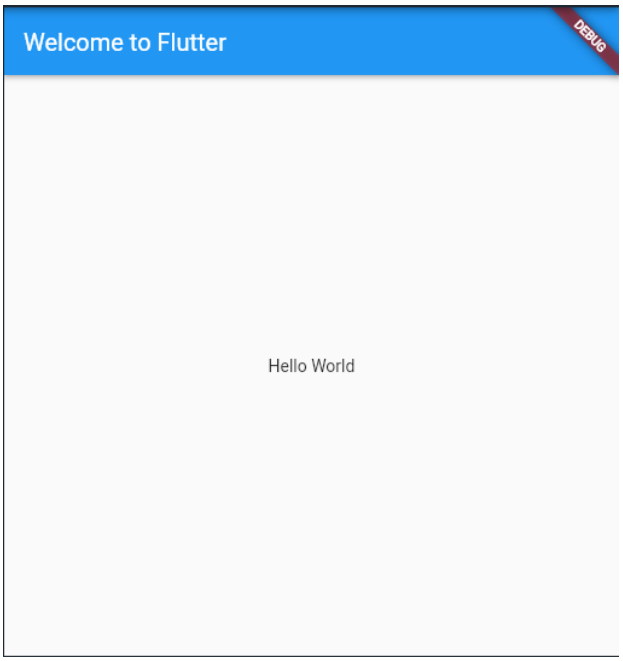


Figure — Sortie du HelloWorldStateless fournie avec le code de cours.

À comprendre

- En Flutter, l'interface est construite comme une arborescence de widgets.
- StatelessWidget représente un widget dont la configuration ne possède pas d'état mutable local.
- La méthode build() décrit l'interface à afficher.
- MaterialApp et Scaffold fournissent la structure générale d'une application Material.

15. Exercice FL2 — StatefulWidget et compteur

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int _counter = 0;

  void _increment() {
    setState(() => _counter++);
  }

  void _decrement() {
    setState(() => _counter--);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter Stateful')),
      body: Center(
        child: Text('$_counter', style: const TextStyle(fontSize: 40)),
      ),
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          FloatingActionButton(
            heroTag: 'minus',
            onPressed: _decrement,
            child: const Icon(Icons.remove),
          ),
          const SizedBox(width: 12),
          FloatingActionButton(
            heroTag: 'plus',
            onPressed: _increment,
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```

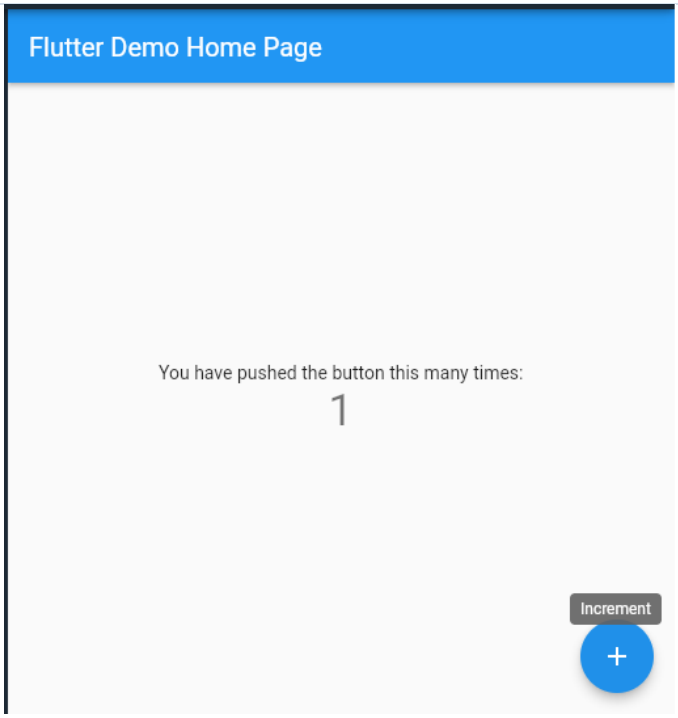


Figure — Résultat du CounterStatefull fourni dans le code du cours.

Lien entre les deux frameworks — setState()
setState() indique à Flutter que l'état a changé et que le widget concerné doit être reconstruit. Cette notion fait écho à this.setState() utilisé dans la partie React Native.

Travail étudiant

1. Ajouter un bouton Reset.
2. Bloquer la valeur minimale à 0.
3. Changer le titre de l'AppBar lorsque le compteur dépasse 5.

16. Exercice FL3 — TextField + ElevatedButton

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: FormPage()));

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
  State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {
  final TextEditingController _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
```

```
  void _showText() {
    showDialog(
      context: context,
      builder: (_) => AlertDialog(
        title: const Text('Texte saisi'),
        content: Text(_controller.text.isEmpty
            ? 'Aucune valeur'
            : _controller.text),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('TextField')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Saisir un texte',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 14),
            ElevatedButton(
              onPressed: _showText,
              child: const Text('OK'),
            ),
          ],
        ),
      ),
    );
  }
}
```

React Native	Flutter
TextInput	TextField
state.input	TextEditingController ou variable de State
Button	ElevatedButton
Alert.alert()	showDialog() + AlertDialog

17. Exercice FL4 — Découvrir Cubit / BLoC

17.1 Ajouter la dépendance

```
flutter pub add flutter_bloc
```

17.2 Code simplifié avec CounterCubit

lib/main.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

void main() => runApp(const CounterApp());

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
}

class CounterApp extends StatelessWidget {
  const CounterApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: BlocProvider(
        create: (_) => CounterCubit(),
        child: const CounterPage(),
      ),
    );
  }
}

class CounterPage extends StatelessWidget {
  const CounterPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter Cubit')),
      body: Center(
        child: BlocBuilder<CounterCubit, int>(
          builder: (context, count) {
            return Text('$count', style: const TextStyle(fontSize: 42));
          },
        ),
      ),
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          FloatingActionButton(
            heroTag: 'minus',
            onPressed: () => context.read<CounterCubit>().decrement(),
            child: const Icon(Icons.remove),
          ),
          const SizedBox(width: 12),
          FloatingActionButton(
            heroTag: 'plus',
            onPressed: () => context.read<CounterCubit>().increment(),
            child: const Icon(Icons.add),
          ),
        ],
      ),
    );
  }
}
```

Élément	Rôle
CounterCubit	Contient la logique et l'état numérique.
emit()	Publie un nouvel état.
BlocProvider	Fournit l'instance de Cubit à une partie de l'arbre de widgets.

Élément	Rôle
BlocBuilder	Reconstruit uniquement la partie UI qui dépend du nouvel état.

À retenir — Différence avec setState

Avec setState(), la logique d'état reste directement dans la classe State du widget. Avec Cubit/BLoC, elle est déplacée dans un composant de logique réutilisable.

18. Exercice FL5 — Accéder à la position GPS avec geolocator

18.1 Installer la dépendance

```
flutter pub add geolocator
```

18.2 Autorisations Android

Dans android/app/src/main/AndroidManifest.xml, ajouter avant la balise <application> :

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

18.3 Exemple Flutter

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';

void main() => runApp(const MaterialApp(home: LocationPage()));

class LocationPage extends StatefulWidget {
  const LocationPage({super.key});

  @override
  State<LocationPage> createState() => _LocationPageState();
}

class _LocationPageState extends State<LocationPage> {
  Position? _position;
  String _message = 'Position non demandée';

  Future<void> _getPosition() async {
    bool enabled = await Geolocator.isLocationServiceEnabled();
    if (!enabled) {
      setState(() => _message = 'Service de localisation désactivé');
      return;
    }

    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
    }

    if (permission == LocationPermission.denied ||
        permission == LocationPermission.deniedForever) {
      setState(() => _message = 'Permission refusée');
      return;
    }
```

```
    final position = await Geolocator.getCurrentPosition();
    setState(() {
      _position = position;
      _message = 'Position obtenue';
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Géolocalisation Flutter')),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: _getPosition,
              child: const Text('Obtenir ma position'),
            ),
            const SizedBox(height: 16),
            Text(_message),
            Text('Latitude : ${_position?.latitude ?? '--'}'),
            Text('Longitude : ${_position?.longitude ?? '--'}'),
          ],
        ),
      ),
    );
  }
}
```

19. Synthèse comparative React Native / Flutter

Critère	React Native	Flutter
Langage	JavaScript / JSX	Dart
UI	Components natifs exposés par React Native	Widgets Flutter
État local	this.state + this.setState()	State + setState()
Architecture d'état	Flux/Redux ou autres bibliothèques	BLoC/Cubit, Provider, etc.
Styles	StyleSheet et Flexbox	Propriétés des widgets, Theme, layouts
Packages	npm / Expo modules	pub.dev / packages Dart-Flutter
Capteur GPS du TP	expo-location	geolocator
Fichier principal	App.js	lib/main.dart

19.1 Travail de réflexion

1. Quel framework vous paraît le plus simple pour construire une interface simple ? Justifier en 3 à 5 lignes.
2. Comparer la notion de state en React Native et en Flutter StatefulWidget.
3. Pourquoi BLoC/Cubit peut-il être intéressant lorsque l'application devient plus complexe ?
4. Comparer les deux codes de géolocalisation : quelles étapes sont communes ?
5. Identifier au moins deux différences entre StyleSheet React Native et la composition de widgets Flutter.

20. Exercices d'approfondissement

Exercice A — Carte d'identité utilisateur

Créer dans les deux frameworks un écran contenant : nom, prénom, groupe, un champ texte et un bouton « Valider ». Après validation, afficher le message « Bienvenue Nom Prénom ».

Exercice B — Compteur enrichi

Ajouter +1, -1, +5, Reset et une couleur différente selon la valeur du compteur.

Exercice C — GPS

Afficher latitude, longitude, précision et un horodatage. Ajouter un bouton « Actualiser ».

Exercice D — Suivi de position (bonus)

React Native : expérimenter watchPositionAsync(). Flutter : expérimenter Geolocator.getPositionStream(). Arrêter l'abonnement proprement lorsque l'écran est détruit.

Exercice E — Architecture

Dans Flutter, déplacer la logique du compteur dans un Cubit. Dans React Native, séparer au minimum le composant d'affichage du composant qui porte l'état.

21. Problèmes fréquents et solutions

Problème	Cause probable	Solution
node ou npm non reconnu	PATH non rechargé	Fermer/réouvrir le terminal et vérifier l'installation Node.js.
Expo ne détecte pas le téléphone	Réseau / Expo Go	Même réseau Wi-Fi, relancer npx expo start, essayer tunnel si nécessaire.
Émulateur Android absent	AVD non créé ou arrêté	Créer/démarrer un appareil dans Android Studio Device Manager.
Module expo-location introuvable	Dépendance non installée	Exécuter npx expo install expo-location puis relancer Expo.
flutter non reconnu	PATH incomplet	Ajouter flutter\bin au PATH puis rouvrir PowerShell.
flutter doctor signale Android licenses	Licences non acceptées	flutter doctor --android-licenses.
Aucun device Flutter	Émulateur arrêté / USB	flutter devices puis démarrer/connecter le device.
Permission GPS refusée	Autorisation utilisateur	Réactiver dans les paramètres de l'appareil ou réinstaller l'application de test.
Position fixe sur émulateur	GPS simulé non configuré	Définir une position dans Extended controls > Location.
Erreur après ajout d'un package Flutter	pubspec non actualisé	flutter pub get, puis redémarrer si nécessaire.

Références pédagogiques et techniques

Supports fournis par l'enseignant :

- Dr Selma Belgacem Ben Mansour, « Développement Mobile : Applications Hybrides-Natives (2) — React Native », ISSAT de Sousse, 2022-2023.
- Dr Selma Belgacem Ben Mansour, « Développement Mobile : Applications Hybrides-Natives (1) — Flutter », ISSAT de Sousse, 2022-2023.

Documentation utilisée pour actualiser les installations et API :

- Expo — Create a project : https://docs.expo.dev/get-started/create-a-project/
- Expo Location : https://docs.expo.dev/versions/latest/sdk/location/
- Flutter — Create a new app / installation : https://docs.flutter.dev/
- flutter_bloc : https://pub.dev/packages/flutter_bloc
- geolocator : https://pub.dev/packages/geolocator